

Thinking Mode

The DeepSeek model supports the thinking mode: before outputting the final answer, the model will first output a chain-of-thought reasoning to improve the accuracy of the final response.

Thinking Mode Toggle and Effort Control

	Control Parameter (OpenAI Format)	Control Parameter (Anthropic Format)	Control Parameter (Responses API Format)
Thinking Mode Toggle ⁽¹⁾	<code>{"thinking": {"type": "enabled/disabled"}}</code>		<code>{"reasoning": {"effort": "none/low/high/max"}}</code>
Thinking Effort Control ⁽²⁾	<code>{"reasoning_effort": "low/high/max"}</code>	<code>{"output_config": {"effort": "low/high/max"}}</code>	<code>}}</code> (<code>none</code> disables thinking mode)

- (1) Thinking mode is enabled by default, with the default effort being `high`
- (2) The mapping between the effort set by the user and the model's actual reasoning effort is as follows:

Requested effort	Actual mapped effort
minimal	low
low	low
medium	high
high	high
xhigh	high
max	max
ultra	max

When using Chat Completion with the OpenAI SDK to set the `thinking` parameter, you need to pass the `thinking` parameter within `extra_body` :

```
response = client.chat.completions.create(  
    model="deepseek-flash",  
    # ...  
    reasoning_effort="high",  
    extra_body={"thinking": {"type": "enabled"}}  
)
```

Input and Output Parameters

Thinking mode does not support the `temperature` , `presence_penalty` , or `frequency_penalty` parameters. Please note that, for compatibility with existing software, setting these parameters will not trigger an error but will also have no effect.

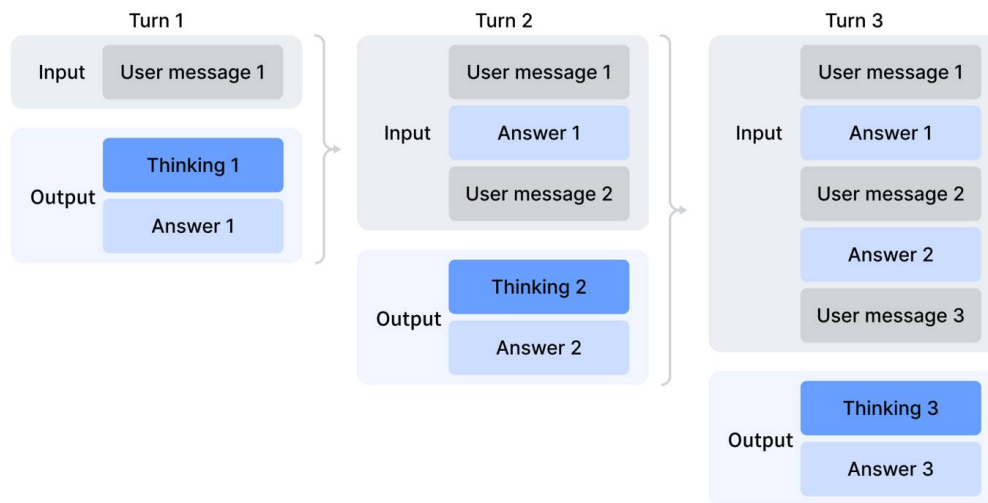
`top_p` only takes effect in thinking mode, where the effective range is `0.95` – `1.0` : values below `0.95` are treated as `0.95` . In non-thinking mode it is fixed at `1.0` and your value is ignored.

In thinking mode, the chain-of-thought content is returned via the `reasoning_content` parameter, at the same level as `content` . In subsequent requests, whether `reasoning_content` should be passed back and whether it will be concatenated into the context depends on whether the request carries the `tools` parameter:

- If the request **carries the `tools` parameter**: the `reasoning_content` of all previous turns should be passed back to the API and will be concatenated into the context. See [Tool Calls](#) for details.
- If the request **does not carry the `tools` parameter**: `reasoning_content` does not need to be passed back; even if passed to the API, it will be ignored and will not be concatenated into the context. See [Multi-turn Conversation](#) for details.

Multi-turn Conversation

In each turn of the conversation, the model outputs the CoT (`reasoning_content`) and the final answer (`content`). If the request does not carry the `tools` parameter, the CoT content from previous turns will not be concatenated into the context in the next turn, as illustrated in the following diagram:



Sample Code

The following code, using Python as an example, demonstrates how to access the CoT and the final answer, as well as how to concatenate context in multi-turn conversations.

[NoStreaming](#) [Streaming](#)

```

from openai import OpenAI
client = OpenAI(api_key="<DeepSeek API Key>",
base_url="https://api.deepseek.com")

# Turn 1
messages = [{"role": "user", "content": "9.11 and 9.8, which is greater?"}]
response = client.chat.completions.create(
    model="deepseek-flash",
    messages=messages,
    reasoning_effort="high"
    extra_body={"thinking": {"type": "enabled"}}
)

reasoning_content = response.choices[0].message.reasoning_content
content = response.choices[0].message.content

# Turn 2
# The reasoning_content will be ignored by the API
messages.append(response.choices[0].message)
messages.append({'role': 'user', 'content': "How many Rs are there in the word 'strawberry'?"})
response = client.chat.completions.create(
    model="deepseek-flash",

```

```

    messages=messages,
    reasoning_effort="high"
    extra_body={"thinking": {"type": "enabled"}},
)
# ...

```

Tool Calls

The DeepSeek model's thinking mode supports tool calls. Before outputting the final answer, the model can perform multiple turns of reasoning and tool calls to improve the quality of the response. The calling pattern is illustrated below:



Please note that for requests carrying the `tools` parameter, the `reasoning_content` must be fully passed back to the API in all subsequent requests — even for turns where the model did not perform a tool call. If your code does not correctly pass back `reasoning_content`, the API will return a 400 error. Please refer to the sample code below for the correct approach.

Sample Code

Below is a simple sample code for tool calls in thinking mode:

```

import os
import json
from openai import OpenAI
from datetime import datetime

# The definition of the tools
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_date",
            "description": "Get the current date",
            "parameters": { "type": "object", "properties": {} },
        }
    },
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get weather of a location, the user should supply the location and date.",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": { "type": "string", "description": "The city name" },
                    "date": { "type": "string", "description": "The date in format YYYY-mm-dd" },
                },
                "required": ["location", "date"]
            }
        }
    },
]

# The mocked version of the tool calls
def get_date_mock():
    return datetime.now().strftime("%Y-%m-%d")

def get_weather_mock(location, date):
    return "Cloudy 7~13°C"

TOOL_CALL_MAP = {

```

```

    "get_date": get_date_mock,
    "get_weather": get_weather_mock
}

def run_turn(turn, messages):
    sub_turn = 1
    while True:
        response = client.chat.completions.create(
            model='deepseek-flash',
            messages=messages,
            tools=tools,
            reasoning_effort="high",
            extra_body={ "thinking": { "type": "enabled" } },
        )
        messages.append(response.choices[0].message)
        reasoning_content = response.choices[0].message.reasoning_content
        content = response.choices[0].message.content
        tool_calls = response.choices[0].message.tool_calls
        print(f"Turn {turn}.
{sub_turn}\n{reasoning_content=}\n{content=}\n{tool_calls=}")
        # If there is no tool calls, then the model should get a final
        answer and we need to stop the loop
        if tool_calls is None:
            break
        for tool in tool_calls:
            tool_function = TOOL_CALL_MAP[tool.function.name]
            tool_result =
tool_function(*json.loads(tool.function.arguments))
            print(f"tool result for {tool.function.name}: {tool_result}\n")
            messages.append({
                "role": "tool",
                "tool_call_id": tool.id,
                "content": tool_result,
            })
            sub_turn += 1
        print()

client = OpenAI(
    api_key=os.environ.get('DEEPSEEK_API_KEY'),
    base_url=os.environ.get('DEEPSEEK_BASE_URL'),
)

# The user starts a question

```

```

turn = 1
messages = [{
    "role": "user",
    "content": "How's the weather in Hangzhou Tomorrow"
}]
run_turn(turn, messages)

# The user starts a new question
turn = 2
messages.append({
    "role": "user",
    "content": "How's the weather in Guangzhou Tomorrow"
})
run_turn(turn, messages)

```

In each sub-request of Turn 1, the `reasoning_content` generated during that turn is sent to the API, allowing the model to continue its previous reasoning.

`response.choices[0].message` contains all necessary fields for the `assistant` message, including `content`, `reasoning_content`, and `tool_calls`. For simplicity, you can directly append the message to the end of the messages list using the following code:

```

messages.append(response.choices[0].message)

```

This line of code is equivalent to:

```

messages.append({
    'role': 'assistant',
    'content': response.choices[0].message.content,
    'reasoning_content': response.choices[0].message.reasoning_content,
    'tool_calls': response.choices[0].message.tool_calls,
})

```

Additionally, in the Turn 2 request, we still pass the `reasoning_content` generated in Turn 1 to the API.

The sample output of this code is as follows:

```

Turn 1.1
reasoning_content="The user is asking about the weather in Hangzhou
tomorrow. I need to get tomorrow's date first, then call the weather
function."

```

```
content="Let me check tomorrow's weather in Hangzhou for you. First, let me  
get tomorrow's date."  
tool_calls=  
[ChatCompletionMessageFunctionToolCall(id='call_00_kw66qNnNto11bSfJVIdlV50o  
, function=Function(arguments='{}', name='get_date'), type='function',  
index=0)]  
tool result for get_date: 2026-04-19
```

Turn 1.2

```
reasoning_content="Today is 2026-04-19, so tomorrow is 2026-04-20. Now I'll  
call the weather function for Hangzhou."  
content=''  
tool_calls=  
[ChatCompletionMessageFunctionToolCall(id='call_00_H2SCW6136vWJGq9SQtBuhVt4  
, function=Function(arguments='{"location": "Hangzhou", "date": "2026-04-  
20"}', name='get_weather'), type='function', index=0)]  
tool result for get_weather: Cloudy 7~13°C
```

Turn 1.3

```
reasoning_content='The weather result is in. Let me share this with the  
user.'  
content="Here's the weather forecast for **Hangzhou tomorrow (April 20,  
2026)**:\n\n☀️ **Condition:** Cloudy \n\n🌡️ **Temperature:** 7°C ~ 13°C  
(45°F ~ 55°F)\n\nIt'll be on the cooler side, so you might want to bring a  
light jacket if you're heading out! Let me know if you need anything else."  
tool_calls=None
```

Turn 2.1

```
reasoning_content='The user is asking about the weather in Guangzhou  
tomorrow. Today is 2026-04-19, so tomorrow is 2026-04-20. I can directly  
call the weather function.'  
content=''  
tool_calls=  
[ChatCompletionMessageFunctionToolCall(id='call_00_8URkLt5NjmNkVKhDmMcNq9Mo  
, function=Function(arguments='{"location": "Guangzhou", "date": "2026-04-  
20"}', name='get_weather'), type='function', index=0)]  
tool result for get_weather: Cloudy 7~13°C
```

Turn 2.2

```
reasoning_content='The weather result for Guangzhou is the same as  
Hangzhou. Let me share this with the user.'  
content="Here's the weather forecast for **Guangzhou tomorrow (April 20,  
2026)**:\n\n☀️ **Condition:** Cloudy \n\n🌡️ **Temperature:** 7°C ~ 13°C
```

(45°F ~ 55°F)\n\nIt'll be cool and cloudy, so a light jacket would be a good idea if you're going out. Let me know if there's anything else you'd like to know!"

tool_calls=None